Fine-grained Evaluation and Validation of Emulator Errors

Christian Krinitsin
Technical University of Munich

Theofilos Augoustis
Technical University of Munich

Sebastian Reimers
Technical University of Munich

Abstract

CPU emulators are used to execute programs across different ISAs, without recompiling them. They are inherently complex by having to support multiple architectures in different combinations and are thus prone to errors. We introduce new extensions to the translation validator VERITAS, a software system used to find and reproduce mistranslation errors in user-mode emulators. We extend the interface to be able to validate BOX64. We extend the bug study to get a better overview of occurring mistranslation errors in QEMU. We also extend the symbolic execution backend Miasm, which is used to create native traces to test against. Last, we extend the minimal reproducer generator in order to be able to validate SQLite.

1 Introduction

Contemporary computer systems are rapidly evolving toward heterogeneity, integrating diverse Instruction Set Architectures (ISAs) like Arm and RISC-V [13, 21]. These emerging platforms offer compelling advantages in performance, energy efficiency, and open hardware ecosystems, but they simultaneously demand flawless execution of the extensive legacy software base [10]. Emulation stands as the principal technique enabling programs to operate across divergent architectures without source modification or recompilation [9, 11, 14, 20, 34, 36]. Beyond ensuring compatibility, emulators are indispensable for dynamic analysis, instrumentation, and developer toolchains. Despite their critical role, constructing correct CPU emulators remains exceptionally challenging, which leads developers to inadvertently miss edge cases, introducing persistent bugs into emulator implementations.

Among the various origins of emulator defects, we concentrate on mistranslation errors, wherein an instruction from the guest architecture is translated into host instructions with divergent semantics. Mistranslation errors surreptitiously corrupt program state and control flow, often manifesting only long after the faulty translation occurs during emulated execution. We argue these errors are particularly severe for correctness, as they undermine every assumption about a program's intended behavior [4, 7, 23, 29, 30].

To show the significance of mistranslation errors, we examine user-mode CPU-emulator bug reports from the past decade for QEMU [14], with a dedicated focus on mistranslation instances. Our analysis of QEMU reveals that mistranslations constitute more than 22% of all user-mode emulator bugs.

Existing testing methodologies do not detect mistranslation errors adequately [2, 17]. Dependence on manually written tests inherently limits detection to the coverage of that specific test suite. While fuzzers [5, 29, 30] automatically generate test programs, expanding coverage, they remain constrained by the fundamental testing paradigm. Consequently, fuzzers—like manual tests—fail to identify mistranslation errors when those errors do not affect the observable output of a generated program. Furthermore, neither manual testing nor fuzzing can reliably pinpoint the source of mistranslation errors within complex emulators.

A system that localizes mistranslation errors is VERITAS. It works for any ISA mapping, across any optimization stages, and reproduces minimal test cases for detected mistranslations. VERITAS treats program execution as state transition systems [31]. A mistranslation is detected if a diversion of a native and an emulated state happens. To model state transition systems, VERITAS relies on the symbolic execution backend Miasm that provides symbolic equations for each state in a program execution.

Figure 1 illustrates an overview of VERITAS. The symbolic execution backend traces the program in order to generate native states and their transitions. VERITAS additionally collects the emulated states and their respective transitions of the tested emulator. We validate the emulated program execution by comparing the appropriate states. If a state mismatch occurs, VERITAS creates a minimal reproducer program that can be repeatedly used as an input.

VERITAS suffers from limitations that we try to eliminate. The symbolic execution backend is not complete as we cannot

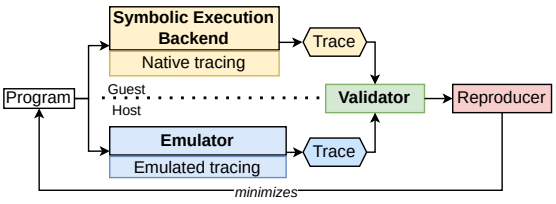

Figure 1: Overview of VERITAS's components.

reproduce all mistranslation errors, as Miasm does not support all instructions. Furthermore, the reproducer generator is not correct at this state: minimal reproducers crash unintentionally and do not reproduce original bugs in all cases.

In this work, we introduce necessary contributions to improve the functionality and quality of VERITAS, which includes:

- An extension of VERITAS' interface that enables the validation of BOX64, a user-mode emulator designed for playing videogames on Arm and RISC-V systems.
- An extension of the existing bug study by categorizing found QEMU mistranslation bugs further, to characterize different types of mistranslation bugs in user-mode emulators.
- An extension of the symbolic execution backend Miasm by implementing new instructions, in order to be able to detect more mistranslations. We will use the extended bug study to methodize this procedure.
- An improvement to VERITAS' minimal reproducer generator in order to support more types of mistranslations and especially large-scale applications like SQLite.

2 Background

Now we will discuss the background for the project. We will take a look at CPU emulators in general, at the foregoing bug study on user-mode emulators, at an example mistranslation error and at the symbolic execution backend and minimal reproducer generator VERITAS uses.

2.1 CPU Emulators

CPU emulators are software systems designed to execute programs compiled for a guest ISA by reproducing their semantics on a different host ISA [30, 36]. This involves providing full support for the guest architecture's instructions, registers, and system behaviors. Typically, CPU emulators are categorized into two fundamental types based on their execution strategy.

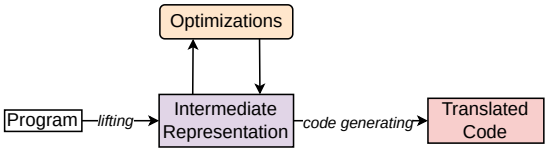

Figure 2: Overview of Binary Translation.

Interpreters operate by fetching, decoding, and emulating a single guest instruction at a time within a central loop. This design is relatively straightforward to implement, but it incurs substantial performance overhead for each instruction, resulting in low overall execution speed.

Binary Translators, in contrast, work by translating discovered blocks of guest instructions into semantically equivalent host instruction sequences. This translated code is written into a cache and executed directly. Although this method is more complex to implement, it eliminates the per-instruction decoding overhead and delivers significantly higher performance.

Modern high-performance emulators commonly employ Dynamic Binary Translation (DBT). The process is illustrated in Figure 2. First, a lifter converts a sequence of guest instructions—referred to as a Translation Block (TB)—into a platform-independent Intermediate Representation (IR). Next, an optimization phase applies various techniques (e.g., constant propagation, dead code elimination) to this IR to enhance its efficiency. Finally, a code generator translates the optimized IR into the host ISA's native instructions. The resulting code block is cached and executed. A key advantage of this multi-stage approach is its modularity; the same optimizer can be reused with different combinations of lifters and code generators for various ISAs [36].

However, this approach introduces considerable complexity. An emulator must integrate lifters and code generators for multiple ISAs, ensuring they function correctly in various combinations and are fully compatible with the chosen IR format. Furthermore, the optimizer must improve performance while rigorously preserving the original semantic behavior of the guest code [36]. Consequently, dynamic binary translators are susceptible to subtle mistranslation errors.

A further important distinction lies in the scope of emulation: system-mode emulators virtualize an entire computer system, including an operating system and all privileged hardware state, while user-mode emulators are designed to execute only unprivileged, user-level application code. The focus of this work is on the latter category, namely user-mode emulation.

2.2 Bug Study

We analyze three user-mode emulators in order to illustrate the significance of mistranslation errors in QEMU [14],

BOX64 [15], and FEX [16]. Using the respective Issue trackers [6–8, 33] we collect all reports from April 2015 to May 2025. We filter out non-user-mode related reports and then classify the remaining user-mode bugs into three error types using the gemma3:27b Large Language Model (LLM).

Emulator	Source	Reports	User mode bugs
QEMU	GitLab [7]	2920	285
	Launchpad [4]	2809	266
	Mailing list [33]	83	0
BOX64	GitHub [6]	1265	1093
FEX	GitHub [8]	856	601
Total		7933	2245

Table 1: Analysis of report sources. *As most reports in the mailing list reference the Launchpad tracking system, the number of mailing list reports is very low. Note that user-mode mistranslations, even if happened in system mode, are still considered a user-mode bug.*

The dataset is illustrated in Table 1. It consists of 7933 reports, with a report's average length of 300 words. Launchpad entries are cross-posted to the mailing list, requiring deduplication.

We categorize the discovered user-mode emulation bugs into three distinct error types: *(i)* Mistranslation Errors occur during the core translation process, when instructions from the guest ISA are incorrectly translated into semantically non-equivalent host instructions. This flaw in the translation logic causes the resulting emulated program to deviate from its intended behavior. *(ii)* System Call Support Errors stem from inaccuracies in the emulation or propagation of system calls. These errors involve the interface between the emulated user program and the host operating system, including incorrect argument handling, improper return value marshaling, or faulty emulation of system call semantics. *(iii)* Runtime Errors form a broader category encompassing all faults that lead to incorrect program execution despite the underlying translation being correct. These errors arise during the execution phase of the translated code, often due to issues in the emulator's runtime environment, such as flawed memory management, incorrect state synchronization, or improper handling of signals and exceptions.

Report quality varies between emulators. While QEMU reports consist of debugging information, BOX64 and especially FEX reports are not diagnostic.

Figure 3 shows the result of the bug study. Within the scope of user-mode bug reports, they account for 22% of all such reports in QEMU and at least 10% in BOX64. It is important to note that for BOX64, this figure represents a conservative lower bound; our categorization may undercount some mistranslation errors due to limited details in certain reports. Additionally, a closer examination of the QEMU data reveals a significant architectural concentration: over 65% of its identified mistranslation errors pertain specifically to

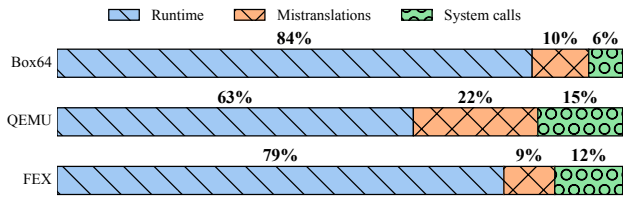

Figure 3: Results of the initial bug study.

either the Arm or x86-64 ISAs. This distribution most likely reflects the popularity of emulation for these widely used architectures, which in turn leads to greater real-world usage and more rigorous testing by the user community, uncovering a correspondingly higher number of latent defects. For FEX, it is important to note that the quality of the reports is so low that we cannot compare the distribution to the other emulators.

2.3 Example of mistranslation

To illustrate the challenges in localizing and fixing mistranslation errors, we examine a bug from QEMU [37] and BOX64 [1]. The mistranslation concerns the x86 `CMPXCHG` instruction that exchanges the value of an operand, if it is equal to the value in `RAX`. Crucially, it does not write the register `RAX` in any case. QEMU and BOX64 incorrectly generates Arm code that zero extends `EAX` to `RAX`, if both operands are equal.

Such a deviation creates an exploitable discrepancy between expected and actual execution, allowing a malicious guest to trigger undefined behavior or cause privilege escalation in a manner that would be impossible on native hardware. This violates the fundamental correctness guarantees of software emulation.

2.4 Symbolic Execution

Miasm is the symbolic execution backend that is used by VERITAS to collect the native traces of a program execution. For a given instruction, Miasm determines symbolic equations and returns them to VERITAS.

In the process of reproducing known bugs in VERITAS, it became apparent that Miasm's specification coverage is low, and it also suffers from similar bugs to the ones from the emulators we analyzed. For instance, QEMU 9.0 lifts the `MOVQ` instruction when used with `MMX` registers incorrectly [25]. VERITAS does not detect this mistranslation due to Miasm suffering from the exact same incorrect behaviour.

For VERITAS to work efficiently, a complete and correct symbolic execution backend is needed.

2.5 VERITAS's Reproducer Generator

After localizing a mistranslation in an emulator execution, VERITAS is able to generate a minimal reproducer program that helps developers with debugging their emulator.

A naive implementation of the reproducer generator proves to be insufficient: only seven out of 15 of detectable QEMU bugs can be minimized, while segmentation faults and unexpectedly large binary sizes are the main reasons for the others not to work. As we also aim to support large-scale applications, an update of the reproducer generator is needed.

3 Overview

We will now give an overview of the different expansions of VERITAS.

The first goal is to implement a new interface to VERITAS, to be able to validate the emulator BOX64. BOX64 has a native tracing functionality that prints a register dump for every emulated instruction in program execution. VERITAS takes the output of BOX64 and creates an internal emulator trace.

Next, we extend the bug study by categorizing all of QEMU's mistranslation bugs into characteristic root causes. Due to the number of found mistranslation bugs being rather low, we conduct the second-level categorization manually.

The expanding of the symbolic execution backend Miasm is necessary in order to increase VERITAS's effectiveness. We need to implement *(i)* the semantics of an instruction in the form of symbolic equations and *(ii)* the decoding of the instruction, such that the binary code is interpreted correctly. We go through the extended bug list by picking a few bugs for every category and implement the missing or incorrect instruction.

The initial implementation of the minimal reproducer generator suffers from two main problems: the segmentation faults an the large binary sizes. The reasons for that is the usage of the `.org` instruction and the `RSP` register. By removing the usage of them, we eliminate both problems.

4 Design and Implementation

Now we will discuss the design and implementation of various parts: The trace of BOX64, the expansion of Miasm and the fixing of the reproducer generator of VERITAS.

4.1 BOX64 Trace

Figure 4 illustrates the output of BOX64's trace feature. We use Regex expressions to parse the output and assign register values to each state. This way, VERITAS collects an emulated trace that can be validated afterwards.

Unfortunately, BOX64 does not support complete traces and we are therefore limited to validating register values only. This partial validation is enough to validate inconsistencies in registers, while not being able to validate Flags or Memory issues. For that, an emulator needs to support complete traces like QEMU does.

The output of BOX64 was initially inconsistent due to a missing whitespace to distinguish between the value and the register. A contribution [3] to BOX64 was made in order to fix this inconvenience.

4.2 Expanding Miasm

We will begin by outlining Miasm's general expansion methodology, followed by a detailed case study to deepen the understanding of this procedure.

General Methodology

Miasm implements instruction decoding in a modular way. Bitfields deal with their own decoding, making use of the values of other bitfields when needed (i.e., the *r/m* bitfield of the *ModR/M* byte, which depends on the value of the *mod* field). For each instruction in binary format, Miasm decides on the most likely fitting instruction candidate and forms the operands. Afterwards, the function that represents the semantics of the instruction is executed.

Instruction semantics are represented using symbolic equations. This means that one needs to find a way to describe the outputs of an instruction, without falling back to the concrete values of the provided input operands. Fortunately, various ISA manuals offer pseudocode specifications [12, 22] of instruction semantics, which we can use as a foundation for our implementation.

Case Study

We will now provide an example of the implementation of the x86-64 `BLSI` instruction, which forms the cause of QEMU-7.2 bug detailed in GitLab Issue 1370 [37].

7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0
$\overline{R}$	$\overline{X}$	$\overline{B}$	m_4	m_3	m_2	m_1	m_0
W	$\overline{v}_3$	$\overline{v}_2$	$\overline{v}_1$	$\overline{v}_0$	L	p_1	p_0

Table 2: VEX encoding bits.

The `BLSI` instruction, as most of the BMI instructions, uses the VEX-prefix, which is missing in Miasm. It provides a lot of useful bitfields (see Table 2), with the following functionalities:

- **W**: This bit changes the operand size from 32 to 64 bits.

Box64's trace output

```
ES=0x002b CS=0x0033 SS=0x002b DS=0x002b FS=0x0043 GS=0x0053
RAX=1234567812345678 RCX=0000000000000000 RDX=0000ffff891feaa0 RBX=0000000012345678
RSP=0000ffff891fea40 flags=???Z?P?
RBP=0000ffff891fea90 RSI=0000ffff891fea90 RDI=0000000077777777  R8=0000000000000000
R9=0000000000000000
R10=0000000000000000 R11=0000000000000000 R12=0000000000401060 R13=0000ffff891feaa0
R14=0000000000000000
R15=0000000000000000 RIP=0000000000401075 0F B1 FB cmpxchg ebx, edi

ES=0x002b CS=0x0033 SS=0x002b DS=0x002b FS=0x0043 GS=0x0053
RAX=0000000012345678 RCX=0000000000000000 RDX=0000ffff891feaa0 RBX=0000000077777777
RSP=0000ffff891fea40 flags=???Z?P?
RBP=0000ffff891fea90 RSI=0000ffff891fea90 RDI=0000000077777777  R8=0000000000000000
R9=0000000000000000
R10=0000000000000000 R11=0000000000000000 R12=0000000000401060 R13=0000ffff891feaa0
R14=0000000000000000
R15=0000000000000000 RIP=0000000000401078 48 8D 3D 81 3F 00 00 lea rdi, [0x0000000000405000]
```

Figure 4: An excerpt of the register dumps before and after BOX64 emulates the `cmpxchg` instruction.

- **R**: This bit expands the *reg* field of the *ModR/M* byte to four bits, so that we can address the new *r8* to *r15* registers.
- **X**: This bit expands the *SIB* (Scale/Index/Base) index field.
- **B**: This bit expands either the *r/m* field of the *ModR/M* byte, or the *SIB* base field, depending on the *Mod* field.
- **m**: This bitfield is an opcode extension. This allows us to distinguish between multiple instructions that share the same opcode.
- **v**: This bitfield is an additional source register. You can therefore have three operand instructions by using this bitfield and the *ModR/M* byte.
- **L**: This bit changes the operand size from 128-bit to 256-bit for vector instructions.
- **p**: This bitfield encodes additional prefix-bytes.

Most of the bitfields, apart from **v** and **L**, are already implemented either by the preceding REX-prefix, which is implemented, or by the base functionality of Miasm. The **L** bit mirrors the **W** bit, with different operand sizes. The **v** bit is the only bit that requires a completely new implementation.

Miasm separates disassemble into a pre-disassembly preparation step and the actual disassembly process. In the pre-disassembly step, Miasm only searches for available prefixes that will be used in the disassembly phase. Actual disassembly involves matching a prefix and generating the disassembled instruction representation. We extend Miasm with a new bitfield module that resembles the **v** bitfield and specify it as an operand for the `BLSI` instruction. The decoding of the bitfield is then handled by existing prefix-matching infrastructure.

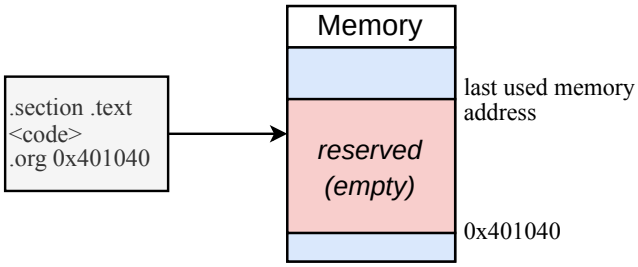

Figure 5: Impact of the usage of `.org` on the memory.

The symbolic equation for the `BLSI` instruction is very simple and it is captured in the pseudocode logic given in the Intel manuals [22]:

$$\text{DEST} = \text{-SRC \& SRC} \quad \text{and} \quad \text{CF} = \text{SRC?(1,0)}.$$

We convert the pseudocode description into symbolic equations manually by describing it in terms of Miasm expressions. Given these symbolic equations, the implementation for `BLSI` is complete and VERITAS can directly use it reproduce the QEMU mistranslation error.

4.3 Reproducer Generator

One of the main changes we made to the reproducer generator was the complete removal of the `.org` instruction usage. The `.org` instruction is used to hard-code addresses in assembly,

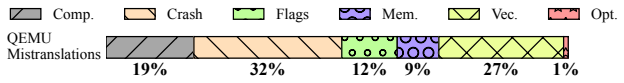

Figure 6: Breakdown of QEMU mistranslation errors.

which makes it possible to have the same value in the same memory address in all invocations. While the use of this instruction sounds helpful and promising, it has a huge flaw: the program reserves all of the memory up to that address for itself, see Figure 5. This leads to *(i)* long linkage times and *(ii)* huge file sizes, surpassing the file size of the original program. We implement a new way to retain memory values: we write necessary values to the `.data` section of the file. To map memory addresses to the same value in each read and write, we calculate the address in memory accesses that follow the form $(base + index) * scale$ and use the address as a variable name.

Another problem is the usage of the `RSP` register. The `RSP` register is the stack pointer in x86-64, i.e., it stores the memory address of the newest value on the stack. It is very important in the context of function calls: when a function is called, the address of the instruction after the function gets pushed on the stack. So the `ret` instruction takes this value from the stack and sets the Program Counter (PC) appropriately. For that it is necessary that `RSP` points to the previously pushed address, after a function's body.

Often, the value of `RSP` will be saved in the mistranslation snapshot that is being reproduced. This leads to the register being set to arbitrary values as a setup that points to local variables in the original program. Ultimately, the minimal reproducer program suffers from a segmentation fault, because the stack frame context gets lost. Additionally, the value of `RSP` is not relevant for any mistranslation, but the values it points to. As they get transferred to the `.data` section when needed, the usage of the `RSP` register is not needed. We disallow setting the register to any values from the snapshot.

5 Evaluation

Here we will evaluate the effectiveness of the different expansions on VERITAS.

After the implementation of the BOX64 interface, VERITAS is now able to validate mistranslations in BOX64. For instance, the bug regarding the `CMPXCHG` instruction that happens in BOX64 0.3.8 is now detectable. Note that we are limited to bugs in combination with registers, as BOX64 has no option to dump the memory.

The breakdown of all QEMU mistranslations can be seen in Figure 6. We found six relevant contexts: Optimizations as a root cause, Vectors/Memory that are used in the faulty instruction, incorrectly set flags, occurring crashes, or regular computation mistakes.

Cat.	Bug ID	Arch.	Emulator	Instruction
Computation	#364.GL [28]	Aarch64	QEMU-5.2	`LDSMAX`
	#508.GL [27]	x86-64	QEMU-6.1	`CMPXCHG`
	#508.Box [1]	x86-64	BOX64-0.3.8	`CMPXCHG`
	#1372.GL [39]	x86-64	QEMU-7.2	`BEXTR`
	#1374.GL [40]	x86-64	QEMU-7.2	`BZHI`
	#1828867.LP [18]	x86-64	QEMU-4.0	`LAHF`
Flag	#1370.GL [37]	x86-64	QEMU-7.2	`BLSI/BLSR`
	#1371.GL [38]	x86-64	QEMU-7.2	`BLSMSK`
	#2175.GL [24]	x86-64	QEMU-8.2	`BLSI`
Vector	#1375.GL [41]	x86-64	QEMU-7.2	`ADDSUBPS`
	#2495.GL [25]	x86-64	QEMU-9.0	`MOVQ`
	#1861404.LP [26]	x86-64	QEMU-4.2	`VMOVDQU`
Crash	#1376.GL [42]	x86-64	QEMU-7.2	`LSL`
	#1377.GL [43]	x86-64	QEMU-8.0	`CVTPS2PD`
	#1832422.LP [19]	x86-64	QEMU-4.0	`CMPPD`
Opt.	#2248.GL [32]	Aarch64	QEMU-8.2.1	`LSR`
Mem.	#2419.GL [35]	Aarch64	QEMU-8.1.3	`LDAPUR`

Table 3: List of known QEMU and BOX64 bug reproducers validated with VERITAS.

Now we can reproduce bugs in each category using a consistent methodology. Table 3 shows all supported bugs in VERITAS.

Figure 7 shows the minimal reproducer sizes for our reproduced bugs. We also support generating a minimal reproducer for the large-scale application SQLiteas a result of our expansion.

6 Future Work

In the future, one could fix the Reproducer Generator further, so that even more bugs can be minimized by VERITAS. For instance, after our expansion, wrong calculations of memory addresses by an emulator cannot be reproduced, because memory addresses are resolved by the generator.

Another problem is the still-incomplete symbolic execution backend. In a new project, a new backend will be constructed, using a formally correct specification language (i.e., ASL, which is used to describe the Arm specification). Such a backend will be correct by construction and is also able to cover the specification of an ISA.

7 Summary

In this work, we expanded VERITAS to improve its effectiveness. We implemented a new interface, so that we are now able to validate the BOX64 emulator. We extended the previously conducted bug study to analyze root causes of mistranslation errors. We expanded the symbolic execution backend of VERITAS, such that many more bugs are now reproducible. We also fixed the reproducer generator and are thus able to create minimal reproducer programs for larger-scale applications, reducing the binary size drastically.

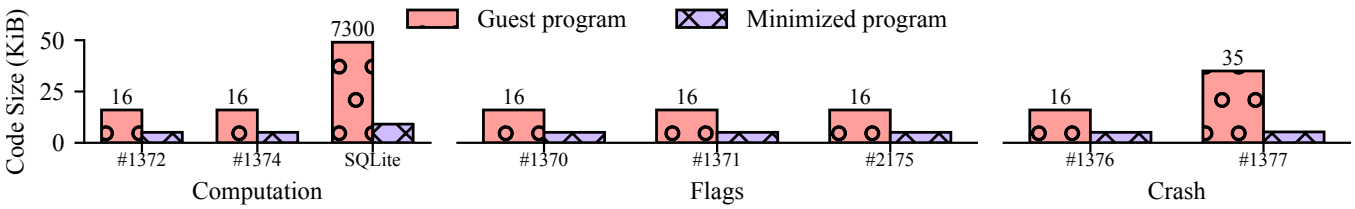

Figure 7: Code size comparison of the original reproducers and the minimal reproducers, generated by VERITAS.

Artifacts

The source code for VERITAS is available at https://github.com/TUM-DSE/focaccia.

References

[1] Box64 Commit log: [dynarec][interp] fixed an edge case of cmpxhg. https://github.com/ptitSeb/box64/commit/70ab16a6d6c54c83a6f9f678c5bbe1c104c6a672. Accessed on: 2025-12-05.

[2] Box64 dynamic recompilation test documentation. https://github.com/ptitSeb/box64/blob/df2e4f941f616a775388004138e605518d9cdb8d/docs/USAGE.md?plain=1#L610. Accessed on: 2025-08-11.

[3] [TRACE] Fix flags output for better parsing. https://github.com/ptitSeb/box64/pull/3090. Accessed on: 2025-10-31.

[4] QEMU Launchpad bug tracker. https://launchpad.net/qemu, 2004-2025. Accessed: August 07, 2025.

[5] Destroying x86_64 instruction decoders with differential fuzzing. https://blog.trailofbits.com/2019/10/31/destroying-x86_64-instruction-decoders-with-differential-fuzzing/, October 2019. Accessed on: 2025-08-11.

[6] Box64 GitHub Issue tracker. https://github.com/ptitSeb/box64/issues, 2020-2025. Accessed: August 07, 2025.

[7] QEMU GitLab Issue tracker. https://gitlab.com/qemu-project/qemu, 2020-2025. Accessed: August 07, 2025.

[8] FEX GitHub Issue tracker. https://github.com/FEX-Emu/FEX/issues, 2023-2025. Accessed: August 07, 2025.

[9] Keith Adams and Ole Agesen. A comparison of software and hardware techniques for x86 virtualization. *SIGOPS Oper. Syst. Rev.*, 40(5):2–13, oct 2006.

[10] Apple. Porting Your macOS Apps to Apple Silicon. https://developer.apple.com/documentation/apple-silicon/porting-your-macos-apps-to-apple-silicon, 2024.

[11] Apple-Silicon. Addressing Architectural Differences in Your macOS Code. https://developer.apple.com/documentation/apple-silicon/addressing-architectural-differences-in-your-macos-co 2020.

[12] Arm Ltd. *Arm Architecture Reference Manual for A-profile architecture*, 2007.

[13] Amazon AWS. Aws graviton processor. https://aws.amazon.com/ec2/graviton.

[14] Fabrice Bellard. Qemu, a fast and portable dynamic translator. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference*, ATEC '05, page 41, USA, 2005. USENIX Association.

[15] The Box64 Developers. Box64 GitHub. https://github.com/ptitSeb/box64/issues/. Accessed on: 2025-07-30.

[16] The FEX Developers. Fex GitHub. https://github.com/FEX-Emu/FEX/issues/. Accessed on: 2025-07-30.

[17] The QEMU Project Developers. Testing in QEMU. https://www.qemu.org/docs/master/devel/testing/main.htmhttps://www.qemu.org/docs/master/devel/testing/main.htmll.

[18] FEX-Emu. QEMU bug report #1828867. https://bugs.launchpad.net/qemu/+bug/1828867. Accessed on: 2025-07-30.

[19] FEX-Emu. QEMU bug report #1832422. https://bugs.launchpad.net/qemu/+bug/1832422. Accessed on: 2025-07-30.

[20] Redha Gouicem, Dennis Sprokholt, Jasper Ruehl, Rodrigo C. O. Rocha, Tom Spink, Soham Chakraborty, and Pramod Bhatotia. Risotto: A dynamic binary translator for weak memory model architectures. In *Proceedings*

of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2023, pages 107–122, New York, NY, USA, December 2022. Association for Computing Machinery.

[21] Agam Shah -- The Register. We're closing the gap with arm and x86, claims sifive: New risc-v cpu core for pcs, servers, mobile incoming. https://www.theregister.com/2021/10/21/sifive_riscv_cpu/, 10 2021.

[22] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual - Volume 2*, 2007.

[23] Muhui Jiang, Tianyi Xu, Yajin Zhou, Yufeng Hu, Ming Zhong, Lei Wu, Xiapu Luo, and Kui Ren. Examiner: automatically locating inconsistent instructions between real devices and cpu emulators for arm. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, pages 846–858, New York, NY, USA, February 2022. Association for Computing Machinery.

[24] Soomin Kim. QEMU bug report #2175. https://gitlab.com/qemu-project/qemu/-/issues/2175. Accessed on: 2025-07-30.

[25] Soomin Kim. QEMU bug report #2495. https://gitlab.com/qemu-project/qemu/-/issues/2495. Accessed on: 2025-07-30.

[26] Stevie Lavern. QEMU bug report #1861404. https://bugs.launchpad.net/qemu/+bug/1861404.

[27] Ilya Leoshkevich. QEMU bug report #508. https://gitlab.com/qemu-project/qemu/-/issues/508. Accessed on: 2025-07-30.

[28] Yves Lhuillier. QEMU bug report #364. https://gitlab.com/qemu-project/qemu/-/issues/364. Accessed on: 2025-07-30.

[29] Lorenzo Martignoni, Stephen McCamant, Pongsin Poosankam, Dawn Song, and Petros Maniatis. Path-exploration lifting: hi-fi tests for lo-fi emulators. *SIGPLAN Not.*, 47(4):337–348, March 2012.

[30] Lorenzo Martignoni, Roberto Paleari, Giampaolo Fresi Roglia, and Danilo Bruschi. Testing CPU emulators. In *Proceedings of the eighteenth international symposium on Software testing and analysis*, ISSTA '09, pages 261–272, New York, NY, USA, July 2009. Association for Computing Machinery.

[31] Robin Milner. *Communication and concurrency*. Prentice-Hall, Inc., 1989.

[32] pjy. QEMU bug report #2248. https://gitlab.com/qemu-project/qemu/-/issues/2248. Accessed on: 2025-07-30.

[33] The QEMU Developers. QEMU mailing list. https://lists.nongnu.org/archive/html/qemu-devel/. Accessed on: 2025-07-30.

[34] Rodrigo C. O. Rocha, Dennis Sprokholt, Martin Fink, Redha Gouicem, Tom Spink, Soham Chakraborty, and Pramod Bhatotia. Lasagne: a static binary translator for weak memory model architectures. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, pages 888–902, New York, NY, USA, June 2022. Association for Computing Machinery.

[35] Eliran Roffe. QEMU bug report #2419. https://gitlab.com/qemu-project/qemu/-/issues/2419. Accessed on: 2025-07-30.

[36] James E. Smith and Ravi Nair. *Virtual Machines: Versatile Platforms for Systems and Processes*. The Morgan Kaufmann Series in Computer Architecture and Design. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2005.

[37] Lee Yong-woo. QEMU bug report #1370. https://gitlab.com/qemu-project/qemu/-/issues/1370. Accessed on: 2025-07-30.

[38] Lee Yong-woo. QEMU bug report #1371. https://gitlab.com/qemu-project/qemu/-/issues/1371. Accessed on: 2025-07-30.

[39] Lee Yong-woo. QEMU bug report #1372. https://gitlab.com/qemu-project/qemu/-/issues/1372. Accessed on: 2025-07-30.

[40] Lee Yong-woo. QEMU bug report #1374. https://gitlab.com/qemu-project/qemu/-/issues/1374. Accessed on: 2025-07-30.

[41] Lee Yong-woo. QEMU bug report #1375. https://gitlab.com/qemu-project/qemu/-/issues/1375. Accessed on: 2025-07-30.

[42] Lee Yong-woo. QEMU bug report #1376. https://gitlab.com/qemu-project/qemu/-/issues/1376. Accessed on: 2025-07-30.

[43] Lee Yong-woo. QEMU bug report #1377. https://gitlab.com/qemu-project/qemu/-/issues/1377. Accessed on: 2025-07-30.